

RISC-V Zbb Bit-Manipulation Extension: ALU Integration on a 5-Stage Pipeline

Your Name

Department of Electrical Engineering

IIT Mandi, Himachal Pradesh, India

your_roll@students.iitmandi.ac.in

Abstract—This paper presents the design and FPGA implementation of the RISC-V Zbb (Basic Bit-Manipulation) extension integrated into a five-stage RV32I pipelined processor. The Zbb subset adds eight instructions — `clz`, `ctz`, `cpop`, `rev8`, `sext.b`, `andn`, `orn`, and `xnor` — as extensions to the existing ALU stage. Two micro-kernels, a `popcount` loop and a CRC-style kernel, are used to evaluate dynamic instruction-count reduction. Post-implementation results on a Xilinx FPGA show an LUT overhead of 8.55%, a flip-flop overhead of 52.07%, and a critical-path penalty of 0.275 ns relative to the baseline RV32I core. The `popcount` kernel demonstrates a 75.86% reduction in dynamic instruction count, directly answering the research question of this project.

I. INTRODUCTION AND MOTIVATION

Modern processor ISAs expose dedicated bit-manipulation instructions to accelerate tasks such as cyclic redundancy checking (CRC), cryptography, compression, and network processing. The RISC-V Zbb extension [1] defines a compact, non-overlapping set of bit-manipulation primitives that can replace multi-instruction software sequences with a single hardware operation.

Without Zbb, operations such as population count (`popcount`) or leading-zero counting (`clz`) require software loops or lookup tables, adding tens of instructions per invocation. Hardware support can collapse these into a single-cycle ALU operation, reducing dynamic instruction count and improving throughput.

Research question: *What is the reduction in dynamic instruction count for bit-manipulation kernels, and what is the ALU critical-path cost of implementing the Zbb extension?*

The remainder of this paper is structured as follows. Section II describes the datapath changes and key design decisions. Section III presents simulation and synthesis results. Section IV provides a direct quantitative answer to the research question.

II. DESIGN AND IMPLEMENTATION

A. Baseline RV32I Pipeline

The baseline processor is a canonical five-stage in-order pipeline with the stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB). Full data forwarding from EX/MEM and MEM/WB to EX resolves most RAW hazards; load-use hazards insert a single stall. Branch resolution occurs at the EX stage with a predict-not-taken policy.

B. Zbb Instruction Subset

The eight Zbb instructions implemented in this work are listed in Table I. All instructions produce a 32-bit result and complete in a single clock cycle within the EX stage; no additional pipeline stages are required.

TABLE I
IMPLEMENTED ZBB INSTRUCTIONS

Mnemonic	Operation	Type
<code>clz</code>	Count leading zeros	R
<code>ctz</code>	Count trailing zeros	R
<code>cpop</code>	Population count	R
<code>rev8</code>	Byte-reverse	R
<code>sext.b</code>	Sign-extend byte	R
<code>andn</code>	AND with complement	R
<code>orn</code>	OR with complement	R
<code>xnor</code>	Exclusive NOR	R

C. ALU Extension

The ALU was extended by adding a Zbb functional unit in parallel with the existing arithmetic, logic, and shift units. A priority MUX at the output selects the Zbb result when the decoded opcode matches any of the eight new instructions; otherwise it passes through the standard ALU output unchanged.

The `clz` and `ctz` units are implemented as a sequence of cascaded comparators using a binary-tree structure to minimize combinational depth. The `cpop` unit uses a parallel adder tree (Wallace-tree-style) to accumulate the 32 bit-lane contributions in $\lceil \log_2 32 \rceil = 5$ adder levels.

D. Instruction Decoding

Zbb opcodes share the `OP` major opcode (0110011) with standard R-type instructions; they are distinguished by their `funct3` and `funct7` fields as specified in the RISC-V ISA specification [1]. The decode stage was extended with additional case entries in the control unit to assert a new `zbb_op[2:0]` control signal passed to the EX stage.

E. Verification

Functional verification was performed in simulation using a directed testbench that exercised all eight instructions across corner cases (all-zeros, all-ones, alternating bits, single set/clear bits). The Zbb-enabled core was also verified to pass all baseline RV32I regression tests without modification.

III. RESULTS

A. Dynamic Instruction-Count Reduction

Two micro-kernels were evaluated:

a) *Popcount loop*: A 32-iteration software loop that counts set bits using `srl/andi` in the baseline was replaced by a single `cpop` instruction in the Zbb implementation. The dynamic instruction count dropped from **29** (baseline) to **7** (Zbb), a reduction of **75.86%**.

b) *CRC-style kernel*: A byte-processing CRC micro-kernel was evaluated as the second benchmark. Both the baseline and Zbb implementations recorded a dynamic instruction count of **19**, yielding **0%** reduction. This result indicates that the chosen CRC kernel formulation does not exercise the currently implemented Zbb instructions on the critical path; instructions such as `rev8` could be leveraged with further kernel optimisation.

Table II summarises the benchmark results.

TABLE II
DYNAMIC INSTRUCTION-COUNT COMPARISON

Kernel	Baseline	Zbb	Reduction
Popcount loop	29	7	75.86%
CRC-style kernel	19	19	0%

B. FPGA Synthesis and Implementation

Both designs were synthesized and implemented for the Xilinx FPGA target available in the VL-326 laboratory. Table III reports resource utilization and timing results.

TABLE III
POST-IMPLEMENTATION RESOURCE AND TIMING COMPARISON

Metric	Baseline	Zbb	Change
LUT	152	165	+8.55%
Flip-Flops (FF)	169	257	+52.07%
BRAM	0.5	0.5	0%
WNS (ns)	4.994	4.719	-0.275 ns margin

C. Critical-Path Analysis

The worst negative slack (WNS) decreased from 4.994 ns (baseline) to 4.719 ns (Zbb), indicating a critical-path penalty of **0.275 ns** introduced by the added combinational logic. Since WNS remains positive in both cases, the design meets timing at the target clock period; the penalty has not yet pushed the design into a timing-violation region.

The `cpop` adder tree and the `clz/ctz` comparator chains are the primary contributors to the increased combinational depth in the EX stage.

D. Area Discussion

LUT overhead of 8.55% is modest and consistent with a small combinational extension to the ALU. The FF overhead of 52.07% is disproportionately large relative to LUT overhead and likely reflects additional pipeline debug registers, counter

logic, or benchmark-instrumentation flip-flops included in the Zbb build. A re-synthesis with all debug logic removed would yield a more accurate architectural FF cost. BRAM usage is unchanged, confirming that the Zbb extension is purely a combinational ALU addition with no memory structures.

IV. CONCLUSION

The RISC-V Zbb bit-manipulation extension was successfully integrated into a five-stage RV32I pipelined processor and validated in both simulation and FPGA implementation.

Answer to the research question: For the popcount kernel, Zbb reduced dynamic instruction count from 29 to 7, a **75.86% reduction**. The CRC-style kernel showed no reduction under the current formulation. The ALU critical-path penalty introduced by Zbb is **0.275 ns**, with LUT and FF overheads of 8.55% and 52.07% respectively. These results confirm that Zbb delivers significant instruction-count savings for population-count workloads at a modest area cost and an acceptable timing penalty that does not violate the target clock constraint.

Future work includes optimising the CRC kernel to exploit `rev8` and `xnor`, profiling the FF overhead after removing debug logic, and evaluating Zbb on a broader set of cryptographic and compression kernels.

REFERENCES

- [1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Version 20191213, 2019. [Online]. Available: <https://riscv.org/specifications/>
- [2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition*, 2nd ed. Morgan Kaufmann, 2020.
- [3] G. Galal and M. Horowitz, "Energy-efficient floating-point unit design," *IEEE Trans. Comput.*, vol. 60, no. 7, pp. 985–1002, Jul. 2011.